

JAVA • REDE DE FLUXO • EDMONDS-KARP

UVA 259 — Software Allocation

Alocação de Aplicações com Fluxo Máximo

Modelagem por rede de fluxo e algoritmo de Edmonds-Karp para verificar se toda a demanda de aplicações pode ser alocada nos computadores disponíveis.

PROBLEMA

Contexto e Objetivo

Entrada

- Aplicações A–Z com demanda de cópias
- Computadores 0–9, compatibilidade variada
- Cada computador executa no máximo uma aplicação

A4 01234 → A precisa de 4 cópias, compatível com 0,1,2,3,4

P2 567 → P precisa de 2 cópias, compatível com 5,6,7

Decisão final

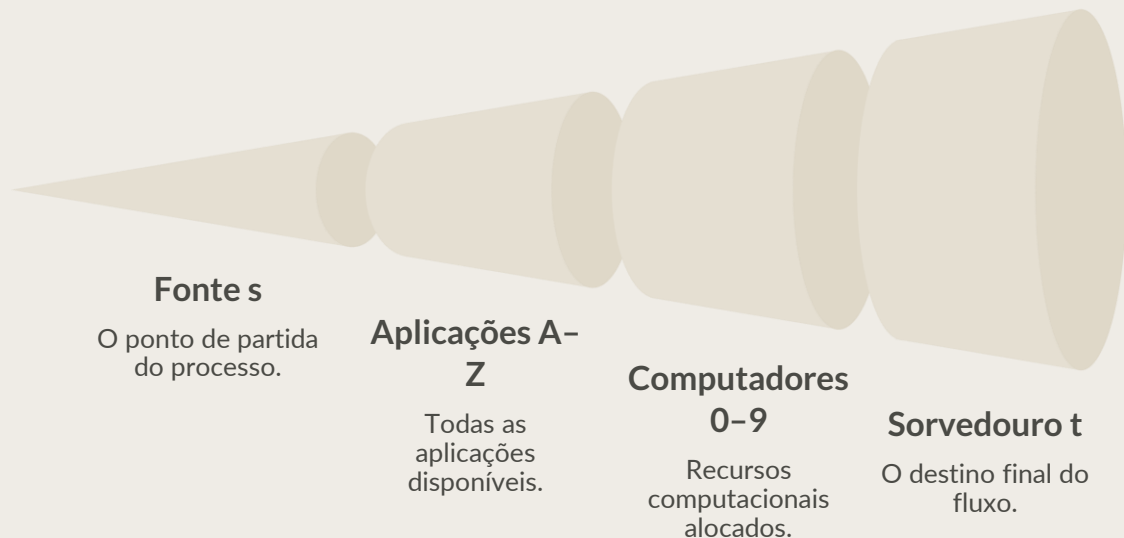
Fluxo máximo == demanda total?

Sim → string de alocação

Não → !

Rede de Fluxo em Camadas

Um único grafo direcionado com quatro camadas. Cada unidade de fluxo representa uma cópia de aplicação alocada em um computador compatível.



O fluxo parte de s , passa pelas aplicações (demanda), pelos computadores (recursos) e chega a t .

Arestas e Capacidades

$s \rightarrow \text{Aplicação}$

Capacidade = demanda

Ex: $A4 \rightarrow s \rightarrow A$ cap. 4

$\text{Aplicação} \rightarrow \text{Computador}$

Capacidade = 1

Ex: A compatível com 0,1,2 $\rightarrow A \rightarrow 0$,
 $A \rightarrow 1$, $A \rightarrow 2$

$\text{Computador} \rightarrow t$

Capacidade = 1

Cada computador recebe no máximo
uma aplicação

Edmonds-Karp e Grafo Residual

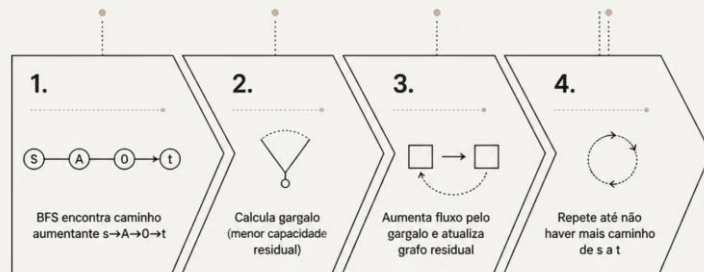
Como funciona

- BFS encontra o caminho aumentante mais curto
- Gargalo = menor capacidade residual do caminho
- Fluxo aumentado pelo valor do gargalo
- Arestas reversas corrigem escolhas anteriores
- Termina quando não há caminho de s a t

Caminho exemplo: $s \rightarrow A \rightarrow \theta \rightarrow t$, gargalo = 1



A classe se chama FordFulkerson, mas usa BFS — o que caracteriza Edmonds-Karp.



SAÍDA

Interpretação da resposta obtida pelo fluxo

Reconstrução da resposta

- $\text{fluxo} < \text{demanda}$ → imprimir !
- $\text{fluxo} = \text{demanda}$ → reconstruir alocação

Ler arestas Aplicação→Computador com fluxo 1.

Computadores livres viram _.

Exemplo de saída

Computadores: 0 1 2 3 4 5 6 7 8 9

Saída: B A _ _ _ _ _ _ _ _

Computador 0 → B, computador 1 → A, demais livres.

Complexidade e Casos Especiais

Complexidade

Edmonds-Karp: $O(V \cdot E^2)$

$V \leq 38 \cdot \text{fluxo máximo} \leq 10 \rightarrow$ abordagem eficiente

Casos especiais

- Múltiplos casos de teste (linha em branco / EOF)
- Demanda total $> 10 \rightarrow$ impossível
- Poucos computadores compatíveis podem bloquear a solução

A modelagem por fluxo máximo transforma o problema de alocação em uma rede com capacidades. Se o fluxo máximo for igual à demanda total, a alocação é possível; caso contrário, não existe solução.

CONCLUSÃO FINAL

O Sorvedouro: A Resposta Final

O nó sorvedouro (t) na rede de fluxo não é apenas o ponto de coleta do fluxo, mas o elemento crucial que valida se a alocação de software é viável ou não.

Fluxo Total = Demanda

A quantidade máxima de fluxo que consegue chegar ao sorvedouro representa o total de cópias de aplicações que puderam ser alocadas nos computadores compatíveis.

Validação da Solução

Se o fluxo máximo de s para t for igual à demanda total de todas as aplicações, a alocação é bem-sucedida. Caso contrário, não há como atender a toda a demanda.

A análise do fluxo no sorvedouro é, portanto, a síntese do problema: uma correspondência direta entre a capacidade da rede e a necessidade do problema real.

Registro de Aprovação

#	Problem	Verdict	Language	Run Time	Submission Date
31175403	259 Software Allocation	Accepted	JAVA	0.120	2026-06-11 01:10:49
					2026-06-01